Mobile Application Development (MAP711S)

Android App Architecture

Simon H. Muchinenyika

Android App Architecture

Table of Contents

- App Architecture Introduction
  - Components, Activating Components, and Architectural Principles
- Android Manifest, App Resources
- The UI Layer
  - Managing State, Activity Lifecycle, The View Model
- The Domain Layer
- The Data Layer

Android App Architecture

Architecture Importance

- improves maintainability, quality, and robustness.
- allows the app to scale.
- helps with onboarding.
- easier to test.
- makes debugging procedural.

Do not reinvent the wheel! Make use of known provable patterns.

However, make room for creativity, innovation.

Android App Architecture

App Components: Activities

- entry point for user interaction.
- though they work together for a single cohesive UX, they're independent of one another.
- modern development follows single-activity architecture.

Discuss benefits & drawbacks.

- activities should be registered in the app's manifest file.
- activities goes through a number of states which are handled by callback methods:

onCreate( ), onStart( ), onResume( ), onPause( ), onResume( ), onStop( ), onDestroy( )

Android App Architecture

App Components: Services

- entry point for long-running background operations.
- once started, continues running in the background even if the user switches to another app.
- a started service runs indefinitely after calling onStartCommand( ).
- a bound service offers a client-service interface by calling bindService( ).
  - it is an implementation of the Service class that calls onBind( ) method.
- When should you use a service and not a thread? Discuss.

Android App Architecture

App Components: Broadcast Receivers

- entry to deliver system-wide events or broadcast announcements.
- follows a publish-subscribe design pattern.

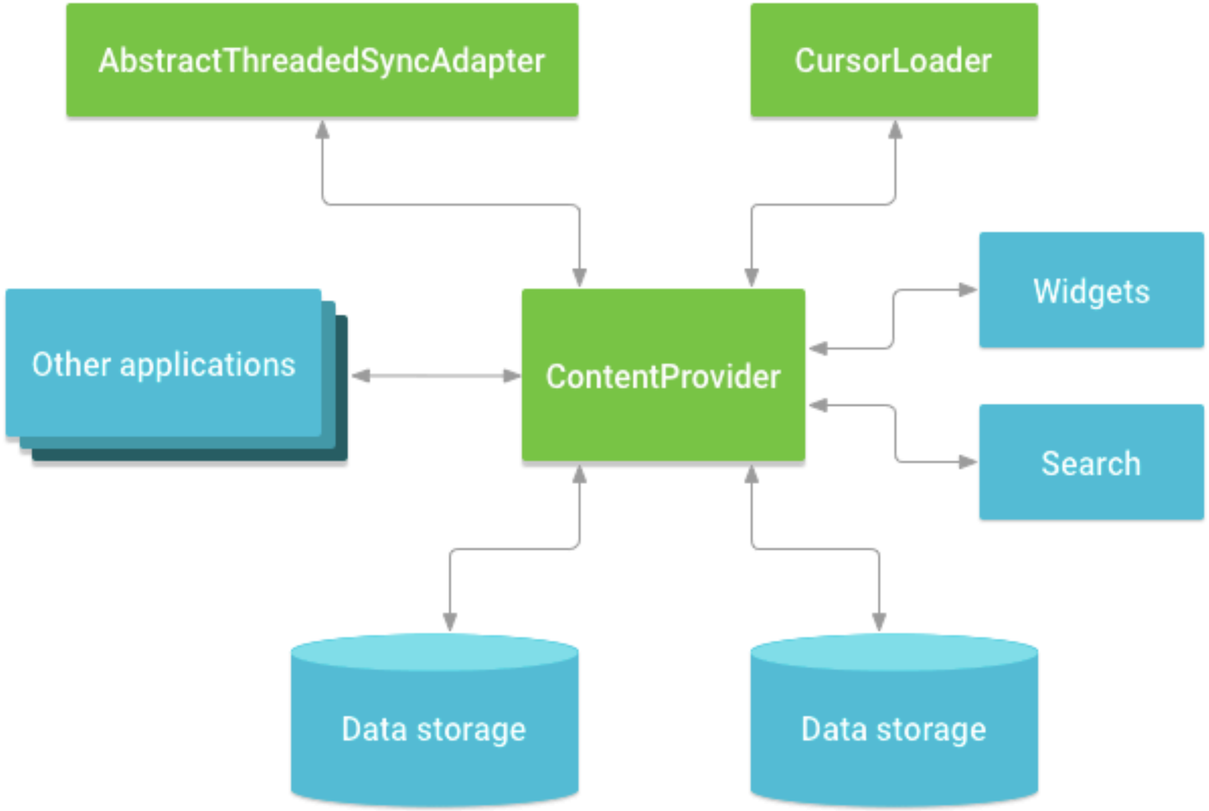
  Example: new data download.
- system broadcasts occur at the system level and are received by all subscribed apps.

  Example: switching in and out of Airplane Mode.
- use context-registered receivers instead of manifest declarations to avoid impacting device performance & user experience.

Android App Architecture

App Components: Content Providers

- entry point for publishing named data items using URIs.
- manages a shared set of app data stored in a file, database, or any other persistent storage.
- use content data to share data with other apps.
- provides access to many components and APIs, see diagram.



Android Activities

Activating Android Components

- asynchronous messages used to activate other Activities, Services, or Broadcasts.
- there are two types of intents:
  - explicit intents

    used with the fully-qualified class name of the target activity.
  - implicit intent

    with general actions to be done on an unspecified target.
- Content Providers are activated using a ContentResolver.

Android Activities

Methods for Activating Android Components

- Activities:
  - startActivity( ) and startActivityForResult()
- Services:
  - startService( ) and bindService( )
- Broadcasts:
  - sendBroadcast( ) and sendOrderedBroadcast( )
- Content Providers:
  - call a query on a ContentResolver.

Android App Architecture

App Manifest

- where all app components are declared:

  <activity>, <service>, <receiver>, <provider>

- read by the Android system before any component is started.
- in addition, the manifest file also:
  - identifies user permissions required by the app.
  - declares the minimum API level.
  - declares hardware and software features required by the app.
  - declares API libraries the app needs to be linked against.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ... >
   <application android:icon="@drawable/app_icon.png" ... >
      <activity android:name="com.example.project.ExampleActivity"
            android:label="@string/example_label" ... >
      </activity>
      ...
   </application>
</manifest>
```

Android App Architecture

App Resources

- Android apps are composed of more than just code:
  - images and audio files.
  - animations, styles, menus, colors, and layouts (defined in XML).
- Separating resources makes it easy to update them without modifying code.
- Alternative resources can also be provided to optimise an app for a variety of device configurations s.a. different languages and screen sizes.
- For every resource, the SDK build tools assign a unique integer ID which is used as a reference in code.

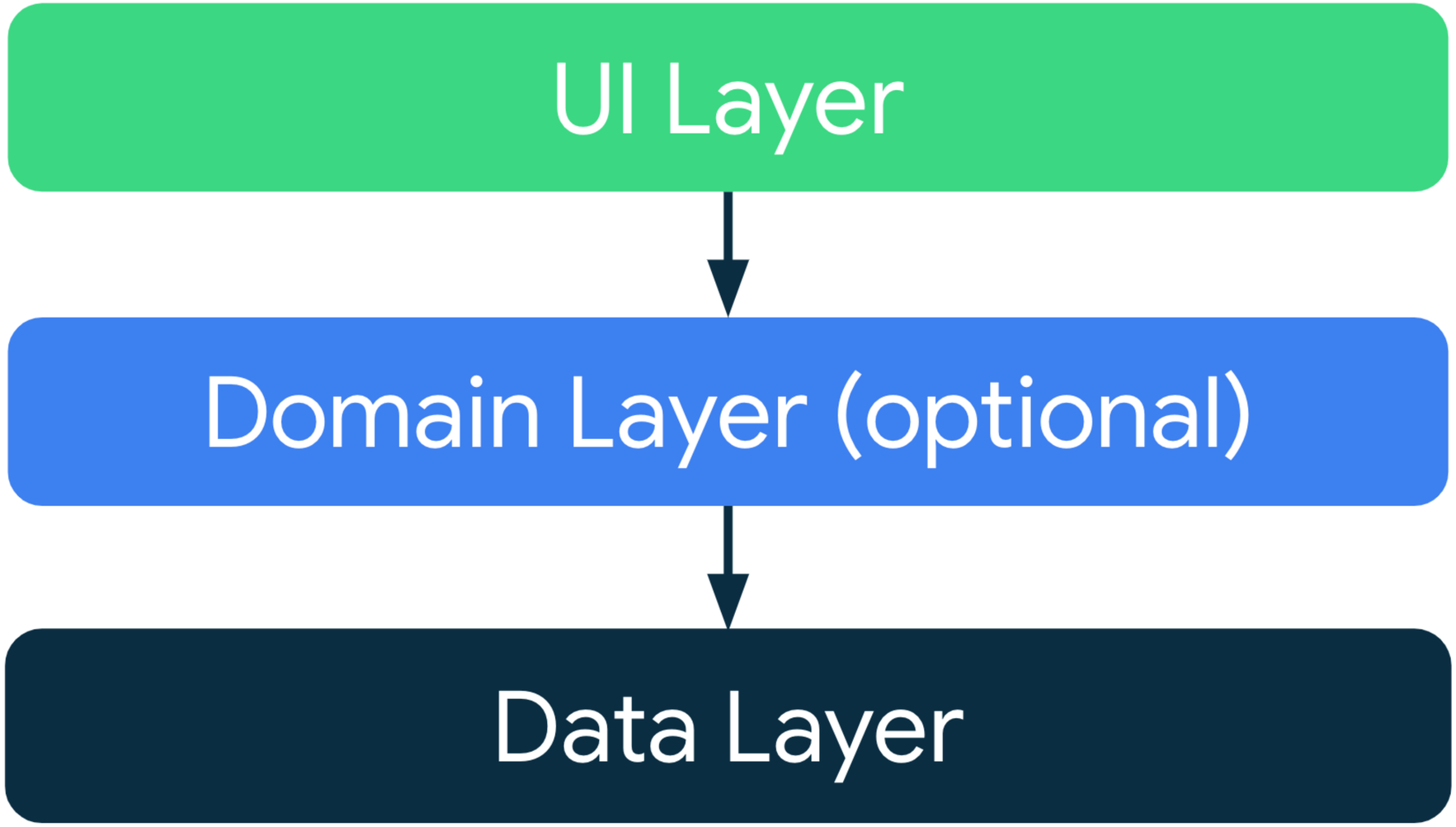
Android App Architecture

Architectural Principles

- Separation of Concerns.
- Adaptive & Layered Architecture.
- Single Source of Truth.
  - Unidirectional Data Flow (UDF) in all layers.
  - UI Layer with State Holder to Manage the UI Complexity.
- Coroutines and Flows.
- Dependency Injection Best Practices.

Android App Architecture

Recommended App Architecture



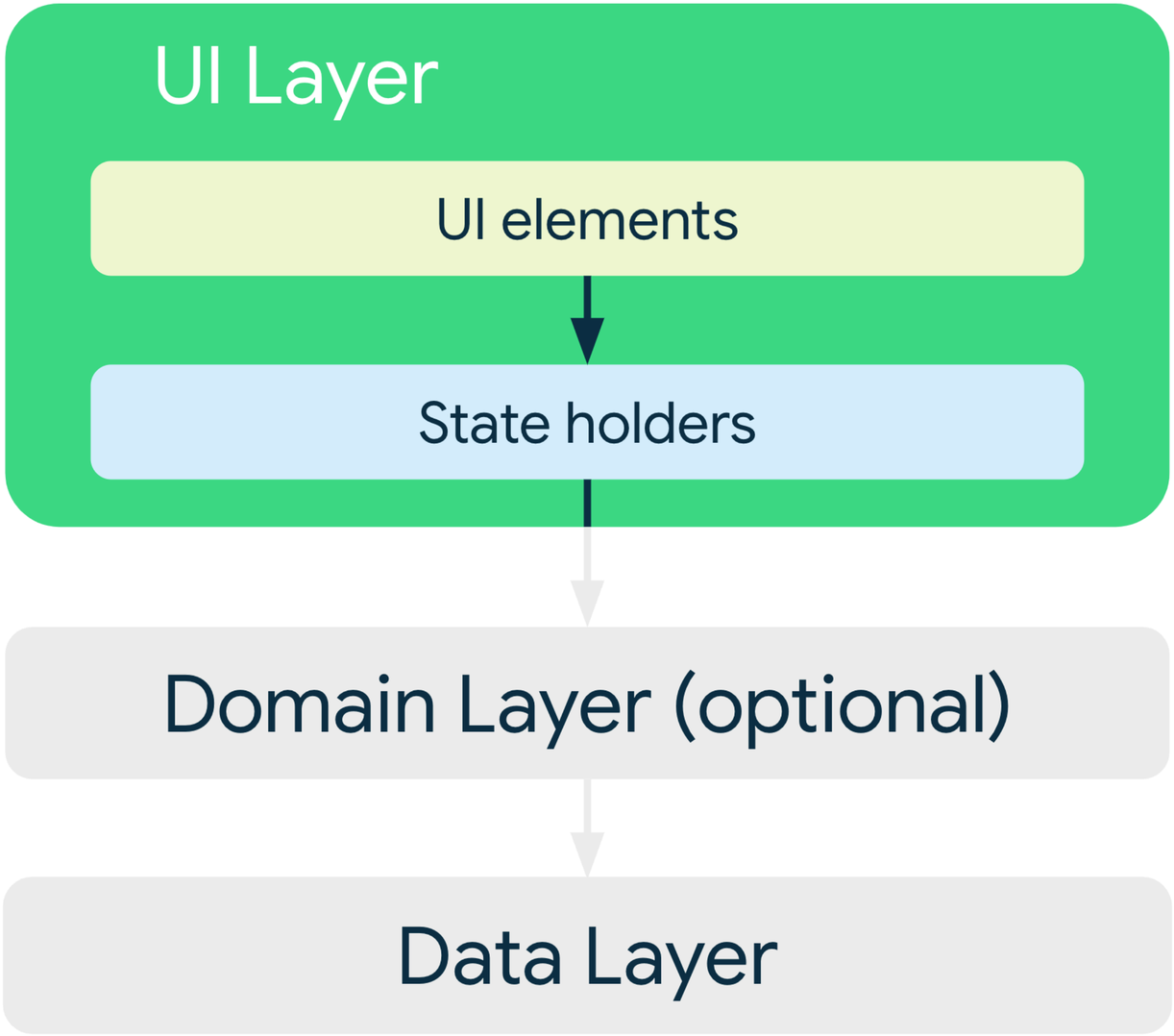
Android App Architecture

UI Operations

- consume app data and transform it into data that the UI can easily render.
- consume UI-renderable data and transform it into UI elements for presentation to the user.
- consume user input events from those user-assembled UI elements and reflect their effects in UI data as needed.
- repeat steps 1 through 3 for as long as necessary.

Android App Architecture

The UI Layer



Android App Architecture

UI Events

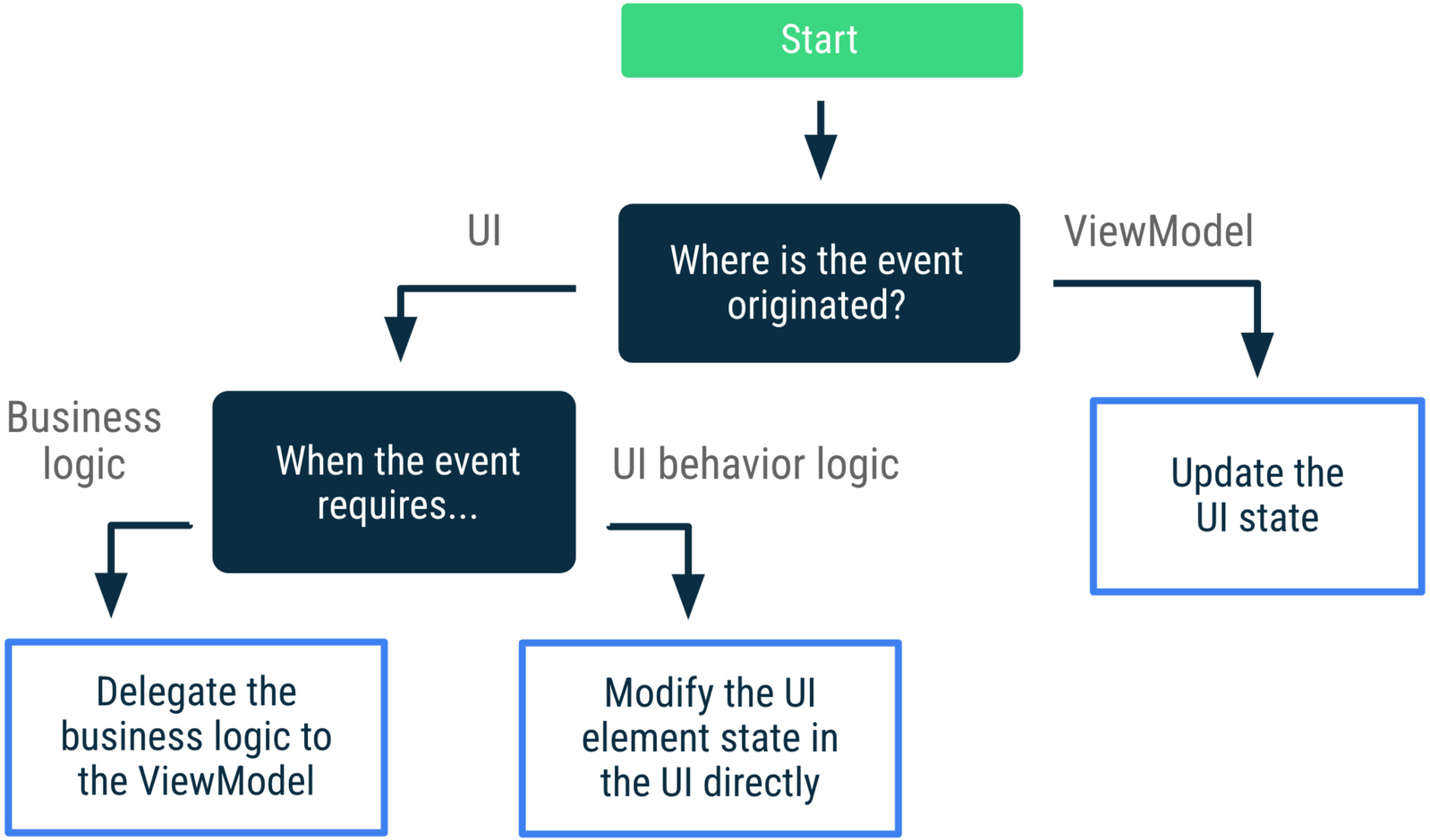
- User Events:
  - clicks, scrolls, gestures, text input.
  - onClick( ),
- Event Listeners:
  - onClickListener, onLongClickListener, onTouchListener, and onKeyListener.
  - setOnClickListener( )
- Event Handling:

onTouchEvent( ), onKeyDown( ), onFocusChanged( )

Android App Architecture

UI Events

- actions handled in the UI, either user actions or by ViewModel.



Android App Architecture

View Binding

- used to write code that interacts with views.
- a binding class with a reference to the root view is generated in that module.
- in the code snippet, the class is ResultProfileBinding, generated from an XML below:

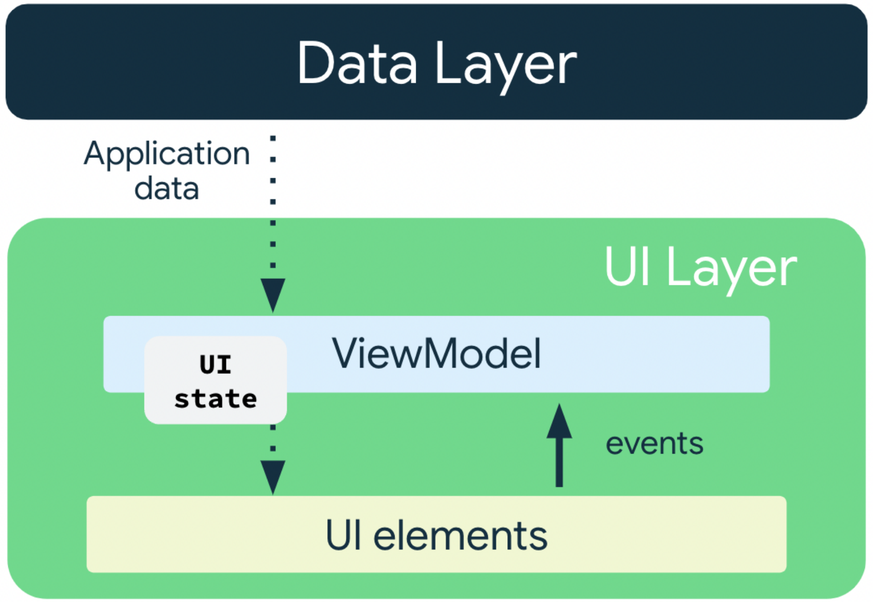
```
<LinearLayout ... >
   <TextView android:id="@+id/name" />
   <ImageView android:cropToPadding="true" />
   <Button android:id="@+id/button"
      android:background="@drawable/rounded_button" />
</LinearLayout>
```

```
private var _binding: ResultProfileBinding? = null
// This property is only valid between onCreateView and
// onDestroyView.
private val binding get() = _binding!!

override fun onCreateView(
    inflater: LayoutInflater,
    container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    _binding = ResultProfileBinding.inflate(inflater, container, false)
    val view = binding.root
    return view
}

override fun onDestroyView( ) {
    super.onDestroyView( )
    _binding = null
}
```

Android App Architecture

The View Model

- derived from the MVVM design pattern.
- abstracts the view by managing and storing UI-related data to separate it from business logic.




```
data class DiceUiState(
    val firstDieValue: Int? = null,
    val secondDieValue: Int? = null,
    val numberOfRolls: Int = 0,
)

class DiceRollViewModel : ViewModel() {

    // Expose screen UI state
    private val _uiState = MutableStateFlow(DiceUiState())
    val uiState: StateFlow<DiceUiState> = _uiState.asStateFlow()

    // Handle business logic
    fun rollDice() {
        _uiState.update { currentState ->
            currentState.copy(
                firstDieValue = Random.nextInt(from = 1, until = 7),
                secondDieValue = Random.nextInt(from = 1, until = 7),
                numberOfRolls = currentState.numberOfRolls + 1,
            )
        }
```

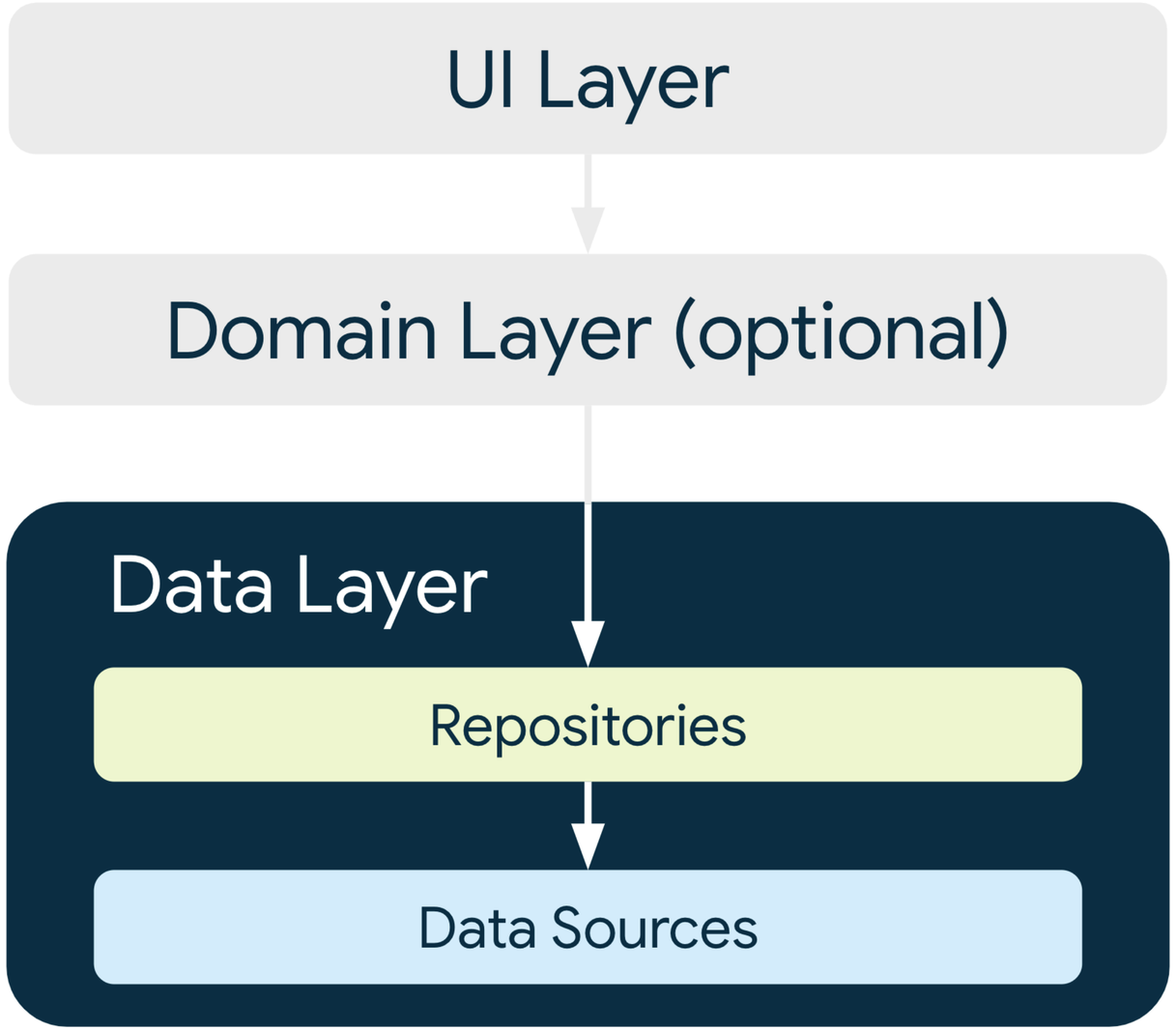
Android App Architecture

Data Layer

- contains application data & business logic.
- business logic contains the rules that determine how application data must be created, stored, and changed.
- application data:
  - data sources may be remote or local, even in memory.
  - Repositories centralises all data sources.
- all data sources should have one source of truth.

Android App Architecture

The Data Layer



Android App Architecture

Repositories

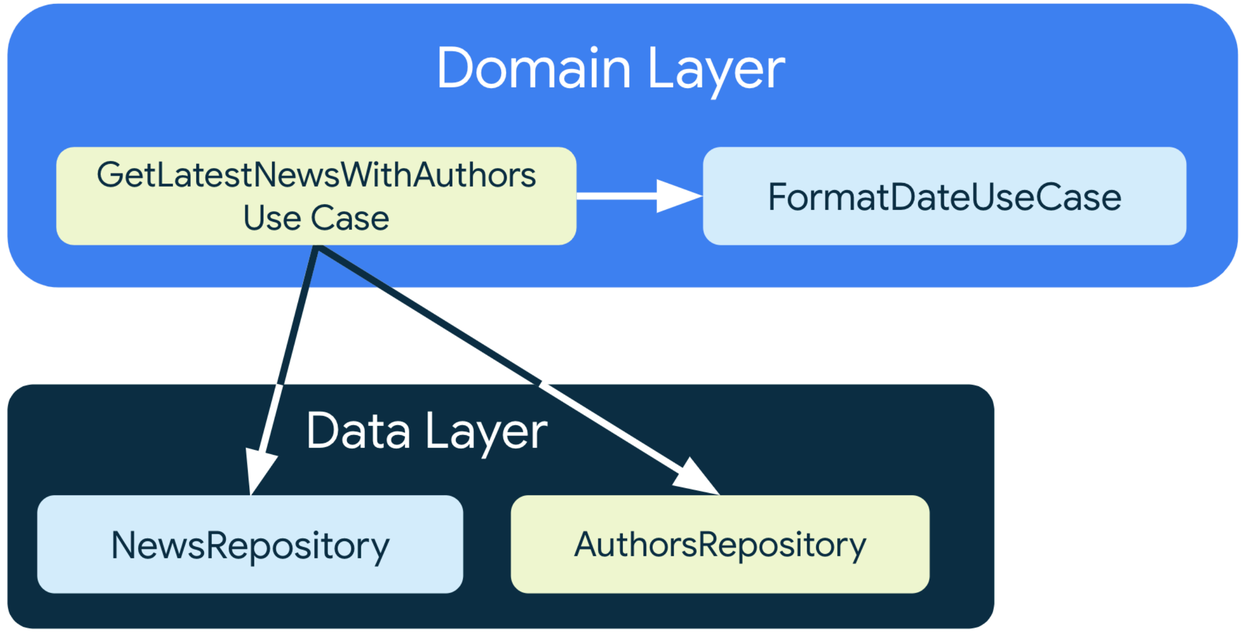
- expose data to the rest of the app.
- centralise data changes.
- resolve conflicts between multiple data sources.
- abstract sources of data from the rest of the app.
- contains business logic.
- Example: dependency injection using immutable data

```
class ExampleRepository(
    private val  exampleRemoteDataSource: ExampleRemoteDataSource, //network
    private val exampleLocalDataSource: ExampleLocalDataSource //database
){/*  ...*/}
```

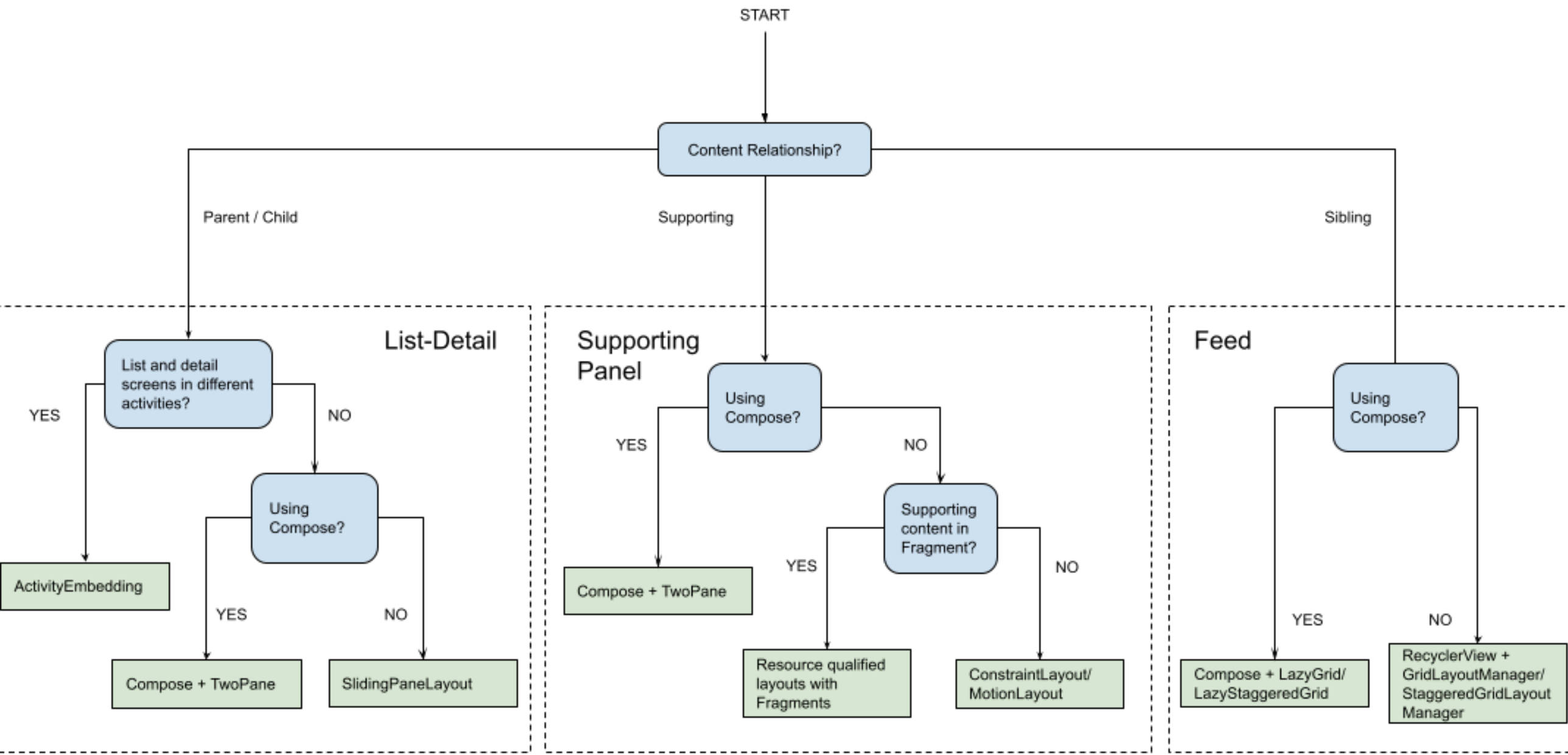
Android App Architecture

The Domain Layer

- encapsulates complex business logic.
- as the app grows, it is essential to add a domain layer.
- it's NOT responsible of how data is displayed - that's for the UI layer.
- neither is it responsible for storing or retrieving data - that's for the Data layer.
- it strictly deals with business logic.
- Example shows some dependencies.



Large Screen Canonical Layout Decision Tree



Android App Architecture

Further Study

- Separation of Concerns:

  https://en.wikipedia.org/wiki/Separation_of_concerns
- Adaptive Layouts:

  https://developer.android.com/develop/ui/compose/layouts/adaptive
- Architectural Samples:

  https://github.com/android/architecture-samples/tree/main
- MVVM:

  https://en.wikipedia.org/wiki/Model–view–viewmodel

Android App Architecture

References

- Android Architecture:

  https://developer.android.com/topic/architecture

- Recommendations:

  https://developer.android.com/topic/architecture/recommendations

- Android UI:

  https://developer.android.com/topic/architecture/ui-layer

- Data Layer:

  https://developer.android.com/topic/architecture/data-layer

Android App Architecture

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na